Three Column Thinking: Four Independent Case Studies — Recorded and Audited

Methodology: E. R. A. Craig (DigitalEuan), Auckland, New Zealand
Geometric Language Machine (GLM) — github.com/DigitalEuan/GLM
Case-study audit and record: AI-assisted independent re-verification
2026-11-01

1. Scope and provenance

This document records four independent Three Column Thinking (TCT) studies — the self-described quartet — produced on **aistudio.xiaomimimo.com** with the updated 14-section TCT skill (the same version evaluated as Run D in the companion paper). The quartet is qualitatively different from the paper's Runs A–D: those were experiment-structured batteries answering fixed questions; these are four self-authored studies that chose their own ambitious open-domain questions, applied the full skill pipeline (decomposition, three columns, Tier-1/Tier-2 verification, ¢-compiler cross-domain mode, optimisation loop, four-conditions runbook), and shipped their own repositories. They are therefore use studies of the skill — evidence of transfer, not just of experimenter-controlled reproduction.

Every study was independently audited for this record (§3): the bundled tooling was diffed against the skill's §13 sources, all Column-3 scripts were re-executed in isolation, ledger claims were re-scanned, optimisation-loop artifacts were re-costed, and — for the first time anywhere in the project — the four Lean tiers were **compiled** on the pure-core toolchain their own `lean-toolchain` files pin (Lean 4.35.0 / Lake 5.0.0).

2. The four studies

2.1 famous-constants-tct — "How can the famous mathematical constants be derived from first principles?"

The answer is a five-class coastline, not a list of formulas: **D** definitional (π, e, φ, √2, Champernowne — definition + algorithm + exact certificate), **T** theorem-pinned (ln 2, ζ(2) = π²/6, ζ(4) = π⁴/90, ζ(3) by Apéry, e^π by Gelfond–Schneider), **O** open (γ — Erdős prize —, ζ(5), closure questions), **U** uncomputable (Chaitin's Ω, the constant

that separates “derivable” from “computable”), and **P** physical (α, G, c/h — **refused** as `WOULD_BE_GUESS`).

Verified highlights: π by three independent derivations (Machin, Euler arctan, BBP) whose exact rational intervals mutually overlap at widths below 10^{-25} with a shared 13-digit block (31415926535897); e by two summation paths with an exact factorial tail; φ with exact Cassini certificates (F_{n+1}² − F_n F_{n+2} = (−1)ⁿ); √2 by integer `isqrt` and Pell certificates ($p^2 - 2q^2 = \pm 1$); ζ(2)/ζ(4) partial-sum sandwiches with telescoping tails; ln 2 by two genuinely different series; Catalan’s constant with alternating certificates; γ bracketed by two independent constructions and stamped `GAMMA_IRRATIONALITY=UNKNOWN` — the open wall kept open, on the record.

Ledger: 24 claims = 13 proved + 3 sorry + 7 axioms + 1 refused. Lean: 6 proved, 3 sorry (two of them deliberate boundary markers: γ’s arithmetic type, α from theory), 7 named axioms. Optimisation loop: ¢ cost 9 → 6 (second iteration rejected — `recip` is term-generation witness machinery); 18 cross-checks written, 18 live after the defect fixes. The verification report documents five assert-layer interventions caught mid-build (truncation anchors inside intervals tighter than the truncation gap, an inverted alternating-remainder parity, a wrong Pell seed sign, Python’s int→str printing limit, an over-wide Mercator bracket) — the protocol catching its own construction errors, which is precisely what the assert layer is for.

2.2 speed-of-light-derivation-tct — “How can the speed of light be derived computationally?”

A question “built to cause issues”, decomposed into four boundaries: **B1** compute the number 299,792,458? — trivially; it is the SI definition (bookkeeping stamped `DEFINITIONAL=1`; light-year 9,460,730,472,580,800 m and ns = 149896229/5000000 cm exact). **B2** derive that light’s speed is the EM wave speed? — yes; historical determinations (Weber–Kohlrausch 1856 vs. Fizeau/Foucault) compared exactly, era agreement under 1/20. **B3** derive that an invariant speed exists? — yes, three unrelated ways: an integer wave scheme whose cone edge speed is exactly 1 cell/step (SUPPORT_T20 = 19), an exact GF(2) symplectic Clifford-circuit propagation with growth ≤ 1 site/layer, and the Lieb–Robinson bound v = 2Ja — which is tight and matches both observed cones; Ignatowski’s causality⇒κ ≥ 0 completes the route. Three unrelated formalisms computing the same cone is the study’s deepest verified result. **B4** derive the dimensionless value (α) from theory? — **no**: `REFUSED: WOULD_BE_GUESS`, with `alpha_from_premises` carried as a deliberate sorry — “the one theorem everyone wants and nobody has”. 1/α computed exactly (floor 137 by two paths) to make the refusal precise about what is checkable.

Ledger: 22 claims = 11 + 3 + 7 + 1. Lean: 5 proved, 3 sorry, 7 axioms. Loop: ¢ cost 10 → 7, witnesses preserved.

2.3 physical-isomorphisms-tct — “How can physical functions be mapped as isomorphisms to computational systems, with measurable improvement — and which hurdles can be overcome?”

The answer disciplines the hype: five measurable gains under five named preconditions — **energy** (wires sum currents “free”: 6419× per MAC-equivalent von Neumann, 69× vs. in-memory), **latency** (propagation replaces sequential steps: 51200 ops → 1 step; direct/FFT convolution = 512/25), **amortization** (setup paid once: optical break-even at reuse T = 21), **sampling** (noise as a free Bernoulli source: exact binomial tail ≤ 1/4; 1% error needs ≥ 2500 bits), **continuous time** (analog ODE evolution: M = 1500 digital Euler steps to match a 1/1000 floor). Every crossover is computed twice (closed form + enumeration) over $\mathbb{Q}/\mathbb{Z}$. And one honest refusal: physics buys no complexity-class speedups — structure-preserving maps preserve hardness (extended Church–Turing carried as a named axiom; the quantum case explicitly open and unclaimed). The hurdle map H1–H8 (data movement, linear algebra, NP-hard search, randomness, ODE/PDE, transport, precision, flexibility) pairs each hurdle with its isomorphism and its caveat — SAT⟺Ising verified exactly (8/8 assignments) with “no proven speedup” stamped beside it.

Ledger: 22 claims = 13 + 3 + 6 + 0. Lean: 9 proved counterparts, 3 sorry, 6 axioms. Loop: ¢ cost 11 → 7.

2.4 maudlin-measurement-trilemma-tct — Maudlin’s three measurement problems

The quantum measurement trilemma — the wavefunction is **complete** (C1), evolves **linearly** (C2), and measurements have **single outcomes** (C3) are individually plausible and jointly incompatible — is re-derived exactly: completeness + linearity force the detector into two orthogonal reading states (`NONZERO_BRANCHES=2`, entanglement witnessed by cross-minors and purity 1/2), contradicting C3 (`C3_CONTRADICTED=1`). The interpretation landscape becomes a truth table: 7 admissible valuations, 1 excluded (111), exactly **three doors** — many-worlds `110` (pays outcomes and probability; `everett_emergence_gap` kept as a deliberate sorry — the Deutsch–Wallace emergence debt, formally on the ledger), objective collapse `101` (GRW: one particle waits 10^{16} s = 12500000000000/39447 years ≈ 3.17×10^{8} yr; a 10^{23} particle object collapses every 1/10000000 s), Bohmian mechanics `011` (adds positions, pays nonlocality). QBism and relational QM are classified as sidesteppers that relocate a claim rather than open a fourth door. The cross-domain mode contributes a fluidic fork feeding a latch that provably cannot select (1/2 − 1/2 = 0) until one extra variable ε breaks the tie — the Bohmian structure in pipes, ledgered honestly as a conjectural analogy.

Ledger: 20 claims = 12 + 3 + 5 + 0. Lean: 10 proved, 3 sorry, 5 axioms. Loop: ¢ cost 12 → 6.

3. Independent audit (this record)

All audit commands and logs ship in the bundle (`audit/` directory). The quartet in aggregate: **88 claims = 49 proved + 12 sorry + 25 axioms + 2 refused** (identities close in all four studies).

Audit check	famous-constants	maudlin-trilemma	physical-isomorphisms	speed-of-light
Bundled §13 scripts byte-identical to skill	4/4	4/4	4/4	4/4
Tier-1 scripts re-run (exit 0)	6/6	6/6	6/6	6/6
Expected-value spot checks (`--key/--expect`)	3/3	4/4	4/4	3/3
`lean_ledger` scan = README claim (sorry/axioms)	3/7 ✓	3/5 ✓	3/6 ✓	3/7 ✓
`cent_cost` v1→v2 = claimed reduction	9→6 ✓	12→6 ✓	11→7 ✓	10→7 ✓
Classification identity n=p+s+a+r	24 ✓	20 ✓	22 ✓	22 ✓
`lake build` **as shipped**	FAILED	FAILED	FAILED	FAILED
`lake build` **after repairs** (below)	OK	OK	OK	OK

Findings.

1. **The Python tier transfers perfectly.** All 16 bundled script copies (4 per study) are byte-identical to the skill's §13 sources; all 24 Column-3 scripts re-verify in isolated subprocesses on this machine, and 14/14 expected-value spot checks taken from the studies' own verification reports (`INTERVALS_OVERLAP=1`, `PURITY=1/2`, `BREAKEVEN_N=55`, `INV_ALPHA_FLOOR=137`, `REFUSAL_REGISTERED=1`, …) match exactly. The ledger, cent-cost and classification claims all reproduce. Nothing in the studies' self-reporting was found inflated.
2. **The Lean tiers do not compile as shipped** — on the very toolchain their own `lean-toolchain` pins (`leanprover/lean4:stable` → 4.35.0, pure core). Three stacked causes, every one of them previously documented by Run D of the companion paper: (i) the scaffold's Lake-4-era lakefile template, which Lake 5.0 rejects at package configuration; (ii) the Mathlib-only notations ℕ and ℚ (47 occurrences across the four studies), which pure core resolves as fresh type variables so every numeral fails `OfNat`; (iii) `decide` used on rational closed arithmetic, which the pure-core kernel cannot reduce. Each study says "`lake build`: not run here; project complete and ready" — the audit shows "ready" was not tested, and is false on pure core.
3. **The minimal repair path is now on record.** With (i) the Run-D lakefile workaround, (ii) ℕ→`Nat` / ℚ→`Rat` substitutions, and (iii) `decide`→`native_decide`,

all four projects build, each carrying exactly its three named sorries as warnings. Caveat recorded: `native_decide` is compiler-trusted, not kernel-checked — the honest route to kernel-checked rationals remains Run D's Nat cross-multiplication pattern (or Mathlib's `norm_num`). The studies' "proved (sorry-free)" Lean counts are therefore compiler-checkable on pure core, not kernel-checked.

4. **The four-conditions runbook transfers.** All four ledgers cite TCT §14, close their classification identities, and record tautology/defect audits; the studies also demonstrate a pattern Run D only glimpsed — the deliberate sorry as a boundary marker (γ, α, `everett_emergence_gap`, `alpha_from_premises`): open problems written into the formal record instead of being hand-waved past.

Interpretation. The quartet independently reproduces the companion paper's central lesson at scale: the skill's Python tier (Column 3 + verify + ledger + cent) is portable, self-verifying, and honest across platforms, while its Lean tier systematically overstates readiness — "proved, not compiled" — until the §8/§9 environment guidance is fixed (probe-first phrasing, Nat-cross-multiplication pattern, Lake-5-compatible scaffold). The audit supplies exactly that missing compilation record.

4. Reproduction

```
# Tier 1 (all four studies, ~seconds):
for d in */; do (cd "$d" && for f in scripts/column3/0*.py; \
  do python3 scripts/verify_column3.py "$f"; done); done
# Ledger scan + loop cost, per study:
python3 scripts/lean_ledger.py lean/tct_<name>
python3 scripts/cent_cost.py --before cent/decomp_v1.json --after cent/decomp_v2.json
# Tier 2 (Lean 4.35): as shipped the build fails at the lakefile; see
# audit/lean_build_audit.log for the named repairs and the successful builds.
```

References

[1] E. R. A. Craig (DigitalEuan), The Geometric Language Machine (Working Paper, revised 21 Oct 2026), github.com/DigitalEuan/GLM. §2.2 (UBP), §2.3 (Three Column Thinking).

[2] `three-column-thinking` Skill (updated, 14 sections incl. the §14 four-conditions runbook), as evaluated in the companion paper.

[3] Companion paper: Three Column Thinking: A Proof-Carrying, UBP-Compliant Reasoning Protocol with Cross-Domain Isomorphic Compilation — A Comparative Skill Study (Runs A–D), 2026.

[4] The quartet repositories: `famous-constants-tct`, `speed-of-light-derivation-tct`, `physical-isomorphisms-tct`, `maudlin-measurement-trilemma-tct` (aistudio.xiaomimimo.com, 2026), bundled verbatim with this record together with the audit logs.